

Bases de Datos NoSQL

MongoDB

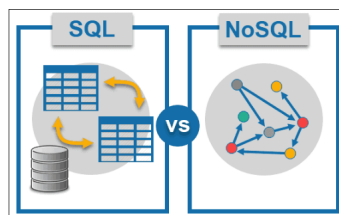
Introducción	3
¿Qué son las Bases de Datos NoSQL?	4
Tipos de bases de datos NoSQL	4
Ventajas de NoSQL	5
¿Qué es MongoDB?	6
MongoDB Shell (JavaScript)	7
Operaciones básicas en MongoDB	9
❖ En el shell de MongoDB	9
❖ En Python (con PyMongo)	11
MongoDB Atlas	14
Herramienta Web – Mongo PlayGround	15
Ejemplo 1 - PyMongo	16
BLOQUE 1:	18
BLOQUE 2	18

Introducción

En Big Data, manejamos volúmenes masivos de datos, que además pueden ser heterogéneos y generarse a alta velocidad (las famosas 5 Vs: Volumen, Velocidad, Variedad, Veracidad y Valor).

Hemos visto que herramientas como Hadoop y Spark nos permiten almacenar y procesar grandes cantidades de datos distribuidos. Sin embargo, para gestionar la persistencia de datos de manera eficiente, necesitamos sistemas de almacenamiento que sean escalables, flexibles y rápidos, algo que las bases de datos tradicionales (relacionales) no siempre pueden ofrecer.

Las bases de datos **NoSQL** (“**Not Only SQL**”) son estos **sistemas de almacenamiento**.



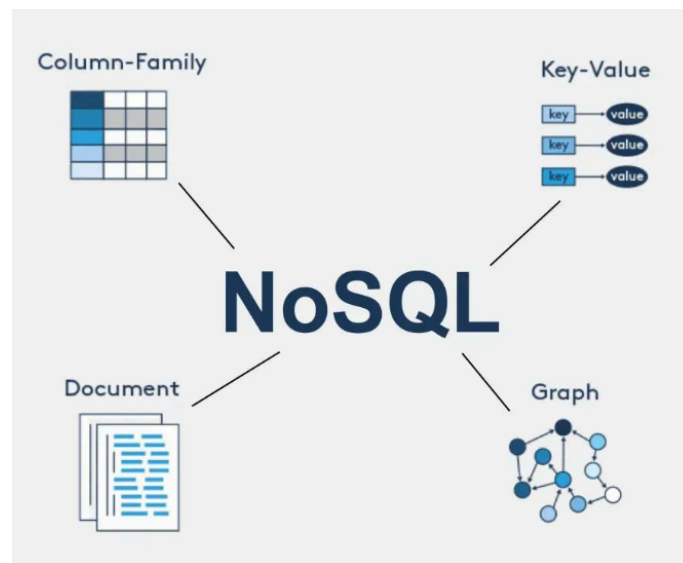
Característica	Base de datos SQL	Base de datos NoSQL
Estructura de datos	Basadas en un modelo relacional y tablas	No están estructuradas o siguen un modelo flexible de documentos, columnas, grafos, etc.
Esquema	Esquema fijo y rígido	Esquema dinámico y flexible
Escalabilidad	Vertical (escalado mediante hardware más potente) u horizontal (escalado mediante la adición de nodos)	Horizontal (escalado mediante la adición de nodos)
Transacciones ACID	Soportadas (Atomicidad, consistencia, aislamiento, durabilidad)	Soportadas parcial o totalmente, dependiendo del tipo de base de datos NoSQL
Consultas complejas	Bien adaptadas para consultas complejas y relaciones complejas entre los datos	Menos adecuadas para consultas complejas y relaciones complejas entre los datos
Flexibilidad	Menos flexible en términos de estructura de datos y cambios en el esquema	Altamente flexible y permite cambios en la estructura de datos sin interrupciones
Escalabilidad lineal	Menor escalabilidad lineal en comparación con NoSQL	Mayor escalabilidad lineal debido a la distribución de datos en múltiples nodos
Costo	Pueden ser más costosas debido a la necesidad de hardware potente y licencias	Pueden ser menos costosas debido a su arquitectura distribuida y la posibilidad de utilizar hardware de menor costo
Ejemplos de sistema	MySQL, PostgreSQL, SQL Server	MongoDB, Cassandra, Redis

¿Qué son las Bases de Datos NoSQL?

NoSQL hace referencia a un conjunto de tecnologías de bases de datos que no utilizan el modelo relacional tradicional. Estas bases están diseñadas para trabajar con grandes volúmenes de datos, distribuidos y de diversa naturaleza.

- ✓ Son bases de datos diseñadas para almacenar y manejar datos de forma flexible, diferente a las bases de datos relacionales tradicionales (SQL).
- ✓ No usan solo tablas fijas con filas y columnas.
- ✓ Pueden manejar datos no estructurados o semiestructurados.
- ✓ Escalan bien horizontalmente (más servidores).
- ✓ Muy usadas en Big Data y aplicaciones web modernas.

Tipos de bases de datos NoSQL



- **Documentales**: almacenan datos en documentos JSON o BSON (ej. MongoDB).
- **Clave-valor**: cada entrada es un par clave-valor (ej. Redis).
- **Columnar**: orientadas a columnas en lugar de filas (ej. Cassandra).
- **Grafos**: diseñadas para representar redes y relaciones (ej. Neo4j).

Ventajas de NoSQL

- Escalabilidad horizontal.
 - Más fácil de escalar agregando nuevos servidores.
 - Soporta grandes volúmenes de datos distribuidos en muchos nodos.
 - Perfecto para aplicaciones que crecen rápido.
- Flexibilidad en el esquema.
 - No necesitas un esquema fijo.
 - Puedes almacenar datos con diferentes estructuras.
 - Ideal para datos no estructurados o semiestructurados (JSON, XML).
- Alto rendimiento en consultas específicas.
 - Muy rápidas en lecturas y escrituras específicas.
 - Optimizadas para consultas simples, como obtener un documento por su clave.
 - Adecuadas para aplicaciones en tiempo real.
- Alta disponibilidad y tolerancia a fallo.
- Ideal para BigData.

¿Qué es MongoDB?



MongoDB es una base de datos **NoSQL** orientada a documentos que permite almacenar, consultar y escalar grandes volúmenes de datos de forma flexible y eficiente.

Características principales de MongoDB:

1. Modelo de datos flexible

- No necesita un esquema fijo: cada documento puede tener diferentes campos.
- Ideal para datos semi-estructurados o con estructuras cambiantes.

Ejemplo de documento:

```
{ "nombre": "Ana", "edad": 28, "ciudad": "Madrid", "hobbies": ["leer", "viajar"] }
```

2. Alta escalabilidad

- Escala horizontalmente agregando más nodos o servidores (sharding).
- Muy usado en entornos de Big Data y aplicaciones web con muchos usuarios.

3. Alto rendimiento

- Optimizado para lecturas y escrituras rápidas.
- Usa índices, replicación y cachés en memoria.

4. Orientado a documentos

- Los datos se almacenan en colecciones (equivalentes a tablas).
- Cada colección contiene documentos (similares a filas, pero más flexibles).

5. Integración con el ecosistema Big Data

- Se integra con Spark, Hadoop, Kafka, etc.
- Permite análisis y procesamiento distribuido.

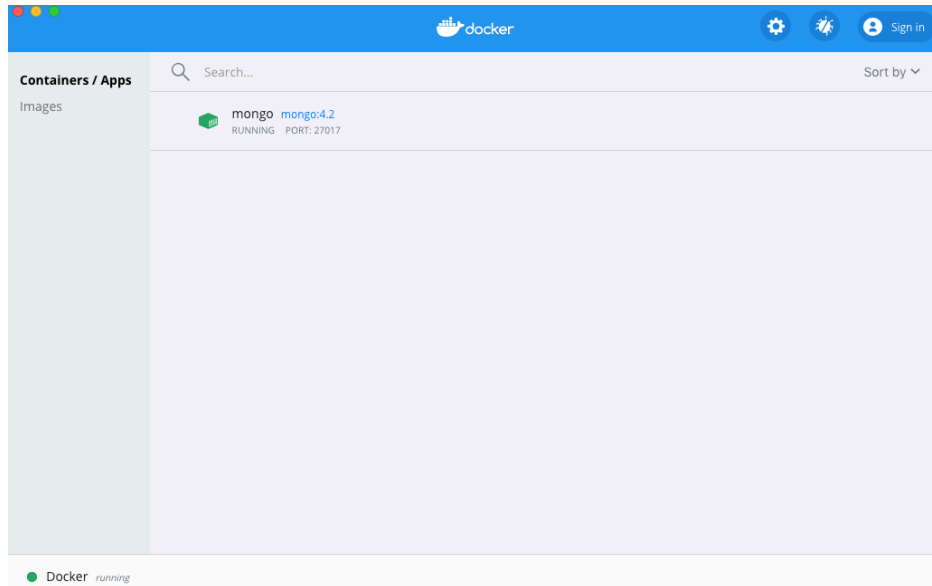
MongoDB tiene librerías para muchos lenguajes:

Lenguaje	Librería común
Python	PyMongo
JavaScript	mongodb (Node.js)
Java	MongoDB Java Driver
C# / .NET	MongoDB.Driver
PHP	mongodb extension
Go	mongo-go-driver

MongoDB Shell (JavaScript)

1. Abrir el Shell del contenedor:

1.1. Si trabajas con un contenedor (docker)



1.1.1. Arranca el contenedor desde el docker

- a. Si no está arrancado el contenedor, hazlo manualmente desde el docker ó puedes ejecutar esta instrucción desde la terminal, cambiando el nombre-de-mi-contenedor por el nombre de tú contenedor:

```
docker run --name nombre-de-mi-contenedor -d -p 27017:27017 mongo
```

Esto inicia un contenedor con MongoDB accesible en el puerto 27017.

1.1.2. Una vez arrancado el contenedor, abre el Shell de MongoDB:

```
docker exec -it nombre-de-mi-contenedor mongosh
```

1.2. Si instalas MongoDB en tu PC (localmente)

1.2.1. Inicia el servidor de MongoDB

Windows (si está como servicio):

```
net start MongoDB
```

Linux/macOS:

```
sudo systemctl start mongod
```

O manualmente:

```
mongod
```

Esto inicia el servidor que escucha por defecto en el puerto 27017.

1.2.2. Abre el shell de MongoDB

Con el shell clásico:

```
mongo
```

Con el shell moderno (recomendado en versiones recientes):

```
mongosh
```

2. Verifica la conexión

```
show dbs // muestra todas las bases de datos
use ejemplo // cambia a otra base de datos
show collections // lista las colecciones de la base de datos que se esta usando
db // muestra la base de datos actual
```


Operaciones básicas en MongoDB

En MongoDB (y en casi todas las bases de datos **NoSQL**) las operaciones básicas se resumen en **CRUD**, que corresponden a los cuatro tipos fundamentales de interacción con los datos:

*Operaciones básicas en MongoDB = **CRUD***

C → **Create**: Se utiliza para insertar nuevos documentos en una colección. (`insertOne`, `insertMany`)

R → **Read**: Permite consultar o recuperar documentos de la base de datos. (`find`, `findOne`)

U → **Update**: Sirve para modificar documentos existentes. (`updateOne`)

D → **Delete**: Para borrar documentos de una colección. (`deleteOne`, `deleteMany`)

A continuación se muestran unos ejemplos de cada método, tanto en la terminal conectándonos al shell de MongoDB como desde Visual Studio Code. Teniendo en cuenta siempre esto:

`db` → se refiere a la base de datos actual.

`usuarios` → es el nombre de la colección (como una tabla en SQL).

❖ En el shell de MongoDB

Primero debes abrir el Shell de MongoDB.

- **Insertar documentos**: MongoDB guarda datos en documentos (JSON-like).

```
db.usuarios.insertOne({nombre: "Ana", edad: 25})  
db.usuarios.insertMany([  
  { nombre: "Luis", edad: 30 },  
  { nombre: "Carla", edad: 28 }  
)
```

- **Buscar/Consultar**: Todos los documentos:

```
db.usuarios.find()
```

Con condición (filtro): “Filtra usuarios con edad mayor de 25”

```
db.usuarios.find({ edad: { $gt: 25 } })
```

Mostrar bonito en la shell: Se usa para mostrar el resultado más “bonito” o legible.

```
db.usuarios.find().pretty()
```

- **Actualizar:**

```
db.usuarios.updateOne({nombre: "Ana"}, {$set: {edad: 26}})
```

- **Eliminar:**

```
db.usuarios.deleteOne({nombre: "Ana"})
```

Resumen Operaciones:

Operación	Método
Crear	insertOne, insertMany
Leer	find, find().pretty(), filtros
Actualizar	updateOne, updateMany, con \$set
Eliminar	deleteOne, deleteMany
Extra	countDocuments, sort, limit

❖ En Python (con PyMongo)

Primero debes establecer la conexión con MongoDB:

- Si nos conectamos al docker

```
from pymongo import MongoClient

# Conéctate al mongodb de tu docker (modifica tu URI en caso de que cambie el puerto)
uri="mongodb://localhost:27017"

client = MongoClient(uri)

# Seleccionar la base de datos y colección
db = client["nombre-de-tu-basededatos"]
usuarios = db["usuarios"]
```

- Si nos conectamos al cluster de MondoDB Atlas

```
from pymongo import MongoClient

# Conéctate a tu cluster (modifica tu URI)
uri="mongodb+srv://<usuario>:<contraseña>@<cluster>.mongodb.net/"

client = MongoClient(uri)

# Seleccionar la base de datos y colección
db = client["nombre-de-tu-basededatos"]
usuarios = db["usuarios"]
```

A partir de aquí ya podemos realizar distintas operaciones:

- **Insertar documentos:** MongoDB guarda datos en documentos (JSON-like).

```
usuarios.insert_one({"nombre": "Ana", "edad": 25})  
usuarios.insert_many([  
    {"nombre": "Luis", "edad": 30},  
    {"nombre": "Carla", "edad": 28}  
)
```

- **Buscar/Consultar:** Todos los documentos:

```
db.usuarios.find()
```

Con condición (filtro): “Filtra usuarios con edad mayor de 25”

```
for doc in usuarios.find({"edad": {"$gt": 25}}):  
    print(doc)
```

En pyMongo no existe el método pretty que se usa para mostrar el resultado más “bonito” o legible. Se puede hacer de esta forma:

```
import pprint  
  
for doc in usuarios.find():  
    pprint.pprint(doc)
```

- **Actualizar:**

```
usuarios.update_one(  
    {"nombre": "Ana"},  
    {"$set": {"edad": 26}}  
)
```

- **Eliminar:**

```
usuarios.delete_one({"nombre": "Aba"})
```

Resumen Operaciones:

Operación	Método
Insertar 1	insert_one({})
Insertar n	insert_many([{}, {}])
Leer todos	for doc in find(): print(doc)
Filtrar	find({"edad": {"\$gt": 25}})
Actualizar	update_one(filtro, {"\$set": {}})
Eliminar	delete_one(filtro)

MongoDB Atlas

Es una plataforma en la nube que permite crear bases de datos MongoDB fácilmente, sin necesidad de instalar nada en local. Proporciona acceso seguro, backups y monitoreo.



Esta es la URL de acceso a MongoDB Atlas:

<https://www.mongodb.com/products/platform/atlas-database>

Tendremos que registrarnos en la plataforma, para poder crear clusters, crear base de datos y colecciones. Pasos:

1. “Get Started” para registrarte (is free!!).
2. “Accept Privacy Policy & Terms of Service”
3. “Submit”
4. Después crea un cluster, una vez creado el cluster crea una base de datos y una colección.
5. Nota: Copia la URI de conexión, algo como esto :

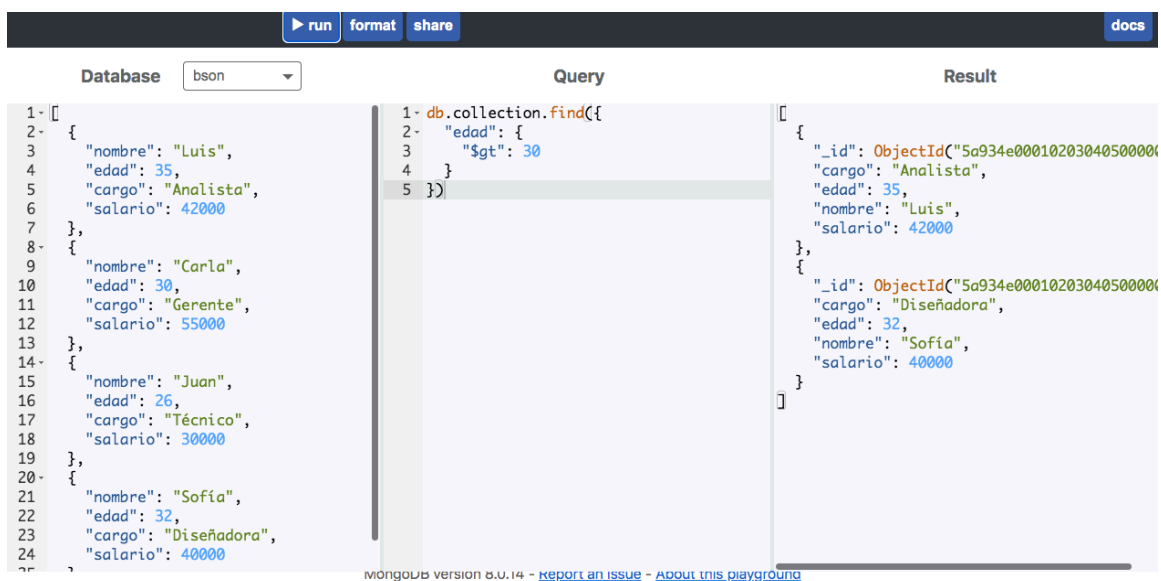
`mongodb+srv://dam2526:SvziVs0VCtG1kXG3@clusterdam.efbbkip.mongodb.net/`

Y a partir de aquí ya podremos desde nuestro IDE conectarnos a MongoDB Atlas para añadir documentos en nuestra colección y consultarlos.

Antes de ver el ejemplo (Ejemplo 1 - PyMongo) de cómo conectarnos a MongoDB Atlas utilizando código Python (PyMongo), vamos a realizar una **práctica** que nos permitirá crear todo lo anterior sin utilizar código!! Haciendo uso solo de la **interfaz web de MongoDB Atlas**.

Herramienta Web – Mongo Playground

- Es una herramienta en línea (web) <https://mongoplayground.net/> que permite escribir, probar y ejecutar consultas MongoDB directamente desde un navegador, sin necesidad de tener MongoDB instalado localmente.
- Proporciona un entorno tipo "playground" (campo de juego) para experimentar con el lenguaje de consultas de MongoDB (por ejemplo find, update, aggregate, etc.).
- Permite definir colecciones de datos de ejemplo (por ejemplo con sintaxis JSON/BSON), ejecutar consultas sobre esos datos y ver los resultados inmediatamente.



Ejemplo 1 - PyMongo

A continuación el ejemplo de código Python (PyMongo) que se conecta a un cluster, selecciona una base de datos y una colección y añade a la colección 3 documentos. Luego realiza unas consultas y actualice un documento.

```
from pymongo import MongoClient

# Cambia aquí tu URI de MongoDB Atlas
# uri = "mongodb+srv://<usuario>:<contraseña>@<cluster-url>/?retryWrites=true&w=majority"
uri="mongodb+srv://dam2526:SvziVs0VCtG1kXG3@clusterdam.efbbkip.mongodb.net/"

# Conectar al cluster
client = MongoClient(uri)
print("Conexión exitosa a MongoDB Atlas")

# Seleccionar base de datos y colección
db = client["miempresa"]
empleados = db["empleados"]

# Insertar 3 documentos en la colección
empleados.insert_many([
    { "nombre": "Ana", "edad": 28, "cargo": "Ingeniera", "salario": 35000 },
    { "nombre": "Luis", "edad": 35, "cargo": "Analista", "salario": 42000 },
    { "nombre": "Carla", "edad": 30, "cargo": "Gerente", "salario": 55000 }
])
print("Documentos insertados en la colección 'empleados'")

# Consulta: mostrar todos los empleados
print("\n Todos los empleados:")
for empleado in empleados.find():
    print(empleado)
```



```
# Consulta: empleados mayores de 30 años
print("\n Empleados mayores de 30 años:")
for empleado in empleados.find({"edad": {"$gt": 30}}):
    print(empleado)

# Actualizar el salario de Ana
result = empleados.update_one(
    { "nombre": "Ana" },
    { "$set": { "salario": 36000 } }
)
print(f"\n Documentos modificados: {result.modified_count}")

# Ver datos después de la actualización
print("\n Empleados después de la actualización:")
for empleado in empleados.find():
    print(empleado)
```

BLOQUE 1:

Utiliza la base de datos “miempresa” creada en MongoDB Atlas y realiza las siguientes operaciones:

- Inserta 5 documentos nuevos a la colección de empleados
- Consulta: Muestra todos los empleados
- Consulta: Muestra solo los empleados mayores de 30 años
- Actualiza el salario de Ana
- Consulta: Muestra los datos después de la actualización

Realiza las operaciones desde código Python (usa PyMongo en tu IDE). Tendrás que instalar PyMongo desde consola con este comando:

```
pip install pymongo
```

BLOQUE 2

1. Define qué es una base de datos NoSQL y por qué surgió esta necesidad.
2. Explica brevemente los cuatro tipos principales de bases de datos NoSQL con un ejemplo de uso.
3. Compara una base de datos relacional con una documental como MongoDB.
4. Realiza un esquema sencillo con tres documentos de ejemplo para una colección de usuarios.